

How I Built My Site With Claude

A step-by-step playbook for media personalities, creators, and multi-brand humans who want to leave Wix behind.

BY KATOYA RAQUELL PALMER

Founder, ToyBox · Author, Five Windows · Writer at katoyapalmer.com

I had three brands, two podcasts, a consulting business, an event series, and one increasingly expensive Wix subscription. This is the exact path I took to move all of it to my own GitHub-hosted, fully analytics-instrumented, search-indexed, mobile-polished, SSL-secured stack — using Claude as my co-pilot. The whole thing now costs me almost nothing to host.

Second edition. Three months on, the sites have grown a tools layer, a blog, and a maintenance problem. All three are in here now.

Inside this playbook

THE STORY Why I left Wix, and what changed in three months

PHASE 1 Plan your brand architecture

PHASE 2 Set up Claude as your co-pilot

PHASE 3 Build the bones — GitHub, domains, DNS

PHASE 4 Design your pages with Claude

PHASE 5 Deploy — and keep deploying **REWRITTEN**

PHASE 6 SEO and search engine discovery

PHASE 7 Install your analytics stack **UPDATED**

PHASE 8 Mobile, accessibility, security **UPDATED**

PHASE 9 Legal pages and consent

PHASE 10 Email and audience

PHASE 11 Real-time visitor pings

PHASE 12 Build real tools with no backend **NEW**

PHASE 13 Add a blog without adding a platform **NEW**

CLOSING What drifts, and how to catch it **NEW**

If you read the first edition: Phases 1–4, 6, 9 and 10 are substantially unchanged and you can skip them. The three that matter now are **Phase 5** — because uploading files stops working once a site is real — and the two new phases at the back.

BEFORE YOU START

This playbook is an account of what I did on my own sites. It is not professional advice, and it is not a guarantee that any of it will work the same way on yours. **Use it at your own discretion, and take the proper precautions.**

That means: back up anything you can't afford to lose before you change it. Don't practice on a live site people depend on. Read what an AI co-pilot writes before you ship it — especially anything touching security headers, privacy language, consent, or analytics. Third-party tools and interfaces change constantly, so check the current documentation rather than trusting a screenshot from September 2026. And if your site is business-critical, handles payments, or sits under a compliance obligation, get someone qualified involved.

ToyBox Consulting & Management, LLC makes no warranty and accepts no liability for outcomes. What you build is yours — including the mistakes.

THE STORY

Why I left Wix — and what happened next

I'm a writer, a systems strategist, and the kind of person who runs more than one thing at once. Over the years that turned into a personal site, a podcast, a Substack, three creative sub-brands, a consulting business, and an event series.

All of it lived on Wix. Wix worked — until it didn't. Every new brand meant another page, another premium template, another monthly bill. Customization was clumsy. SEO felt locked in. Real-time analytics was a paid add-on. Migrating anything was a fight.

I moved everything to GitHub Pages with Claude as my co-pilot. Hosting is functionally free. I own the code. I can change anything I want.

What the first edition didn't know

I wrote edition one in June, about a month after launch. A site that is one month old is still a brochure. Three months in, three things happened that the first edition has nothing to say about.

It grew a tools layer. There are now eight interactive diagnostics on the consulting site — scored assessments that run entirely in the visitor's browser, with no server, no sign-up, and no data leaving their device. That turned out to be the single most valuable thing on the site, and it cost nothing to run. Phase 12.

It grew a blog. GitHub Pages runs Jekyll natively, which I did not know in June. Seven posts, real permalinks, an RSS feed, no platform and no subscription. Phase 13.

It started to drift. A stale banner on sixteen pages. Two versions of the same page, both in the sitemap. A finished tool nothing linked to. A page of unfilled placeholders, live. None of that is a Wix problem or a GitHub problem — it is what happens to any site nobody audits. The closing chapter is about catching it, and it is the most useful thing I have learned since June.

End state: three live sites, eight browser-native tools, a Jekyll blog with an RSS feed, four analytics tools, a hand-written security policy on every page, and a hosting bill of roughly zero.

PHASES 1–4 · UNCHANGED FROM THE FIRST EDITION

Architecture, co-pilot, bones, design

Phase 1 — Plan your brand architecture

A media personality usually isn't one brand — they're a constellation. Decide before you write a line of code. My model is one hub and many spokes: **katoyapalmer.com** is the hub, each sub-brand is a page on that same domain, **toyboxconsulting.net** is a separate domain and repo for client-facing work, and event sites are standalone landing pages that go live and retire without cluttering anything.

Phase 2 — Set up Claude as your co-pilot

You don't have to know how to code. You need to describe what you want, and a co-pilot that can write, click and verify on your behalf. I use **Claude Cowork** for the work that touches my actual files and accounts, and **Claude Design** for generating on-brand pages from a prompt. Connect the accounts you'll touch: GitHub, Gmail, Calendar, Drive.

Phase 3 — Build the bones

One GitHub repo per site, under an organization rather than a personal handle, so the repos read as a business asset. Public repos are required for free Pages hosting. Buy the domain, then point four A records at GitHub's IPs and add a CNAME for **www**.

Phase 4 — Design your pages with Claude

Describe a feeling, get a page. Claude returns a self-contained HTML file with the CSS and any JS inline — no build step, no framework. Iterate in plain English until it lands, then drop it in the repo.

What I'd add now: ask for a **multi-artboard design canvas** rather than one page at a time. Seeing the home page, an interior page and a mobile view side by side catches inconsistencies that reviewing them one at a time never will.

PHASE 5 • REWRITTEN

Deploy — and keep deploying

The first edition told you to click **Add file** → **Upload files** and drag your HTML in. That is still the right way to launch. It is the wrong way to live.

Once a site is real, most changes are not "replace a page." They are "change one line on every page" — a phone number, a nav item, a logo filename, a banner that expired. Dragging twenty-five files into a browser to change one word each is how sites rot.

The thing nobody tells you: press the full stop key

Open any GitHub repo and press **.** — or change `github.com` to `github.dev` in the URL. You get **Visual Studio Code, running in your browser**, already signed in to your repo. Nothing to install.

What that unlocks:

- **Find and replace across every file at once.** `Ctrl+Shift+F`. Change one string in twenty-five files in a single action.
- **A real diff before you commit**, in the Source Control panel, so you can see exactly what changed.
- **Commit and push from the same screen.**

I recently changed a logo filename in eighteen files, added a missing link to seven more, and fixed footer wording in ten — twenty-five files, forty-five lines — in about four minutes.

Two traps, both of which caught me. VS Code's search is **case-insensitive by default**: searching for "Services & rates" also found a lowercase "services & rates" I did not want capitalized. Turn on match case (the **Aa** button) before every Replace All. And the search box **grows taller** for long strings, pushing the Replace field down the panel — so a click aimed at Replace lands back in Search.

The discipline that makes it safe

Before any bulk replace, know the number. Count the occurrences you expect — Claude can do this against a local copy — then check the **"N results in M files"** that VS Code reports before you commit to the replace. If the numbers disagree, stop. Mine disagreed once, and that is exactly how the case-sensitivity trap got caught instead of shipped.

Remember what "push" means here. GitHub Pages serves your main branch. There is no staging step. The moment you commit, it is live.

PHASE 6

SEO and search engine discovery

Per page: a `<title>` of 50–60 characters, a meta description of 140–160, Open Graph and Twitter Card tags, JSON-LD schema, a canonical URL, and a favicon. Site-wide: `sitemap.xml`, `robots.txt`, and both submitted to Google Search Console and Bing Webmaster Tools.

The sitemap is not a set-and-forget file. Mine drifted badly. It listed a duplicate page I had forgotten existed, and it was missing every blog post and my entire partner-recruitment page. A sitemap that doesn't match your site is worse than none — it tells Google to index the wrong things. Re-check it every time you add a page.

PHASE 7 • UPDATED

Install your analytics stack

Four tools in parallel, each answering a different question. All snippet-based — paste into `<head>` and you're done.

TOOL	THE QUESTION IT ANSWERS
Cloudflare Web Analytics	How many people visited? Privacy-first, no cookies.
Microsoft Clarity	Why did they bounce? Session recordings and heatmaps.
Google Analytics 4	How does this connect to my campaigns?
A B2B identifier	Which <i>companies</i> visited? (Albacross, RB2B, and others.)

Added since the first edition: a live chat widget (I use Crisp), and Cloudflare Turnstile on every form — a CAPTCHA replacement that most visitors never see.

Learn from my mistake. I trialled a second B2B identifier without fully removing the first, and ended up running **both** — one on twenty-eight pages, one on nine. Two vendors doing the same job, two sets of cookies, two entries owed to my privacy policy, and I only found it by auditing. Every script you add is a permanent tenant. Audit the list quarterly and evict what you no longer use.

PHASE 8 • UPDATED

Mobile, accessibility, security

Mobile. Over half my traffic. One `@media (max-width: 820px)` block per page, tested on an actual phone. Bigger headlines, single-column stacking, larger tap targets.

Accessibility. Alt text on every image, 4.5:1 contrast on body text, real `<button>` elements, `lang="en"` on the html tag.

Security headers — worth more effort than you think

A hand-written **Content-Security-Policy** now sits on all thirty-one of my pages. It lists, explicitly, every external host allowed to run a script, load a font, or open a frame. It is tedious to write once and then it silently blocks an entire category of attack forever.

Alongside it: a **Permissions-Policy** switching off camera, microphone, geolocation and payment APIs I never use, and **Referrer-Policy: strict-origin-when-cross-origin**.

The five-minute one nobody does: add `/.well-known/security.txt` — an RFC 9116 file saying how to report a vulnerability to you. Costs nothing. Signals that someone is home.

PHASES 9–11 • UNCHANGED

Legal, email, and knowing who showed up

Phase 9 — Legal. Privacy Policy, Terms of Use, Cookie Notice, plus a consent banner. Claude drafts all three in one pass if you feed it your URL, your analytics list and your jurisdiction. Link them in every footer. Have a lawyer review if you take payments.

Phase 10 — Email. I use Brevo: 300 emails a day free, unlimited contacts. The boring part is the DNS — SPF, DKIM and DMARC records — and it is the part that decides whether you land in the inbox or the spam folder. Your platform generates them; paste them into your registrar.

Phase 11 — Visitor pings. The one Wix feature I missed. A B2B identifier plus a Slack integration puts a message in a channel when a known company hits the site. Pair it with Clarity's alerts for high-friction sessions.

PHASE 12 · NEW IN THIS EDITION

Build real tools with no backend

This is the part I would tell June-me to do first.

Everyone's site has a contact form. Almost nobody's site has something a visitor can *use*. I now have eight scored diagnostics — an operational health check, a grant-readiness assessment, a board governance self-assessment, an operating-reserve calculator, and others. A visitor answers the questions, gets a live score, sees where they're weak, and can print a report.

They cost nothing to run, because **there is no server**.

How it actually works

- **All the logic is in the page.** The questions, the weighting, the scoring, the recommendations — plain JavaScript in a single HTML file. Same as any other page in your repo.
- **State lives in `localStorage`**. Answers persist if the visitor closes the tab and comes back. Seven of my eight tools do this.
- **Nothing is transmitted.** No database, no account, no cookie banner needed for the tool itself. You can say plainly: *nothing leaves your browser* — and mean it.
- **Printing is free.** A `@media print` block turns the results into a clean branded report. No PDF library.

The one thing that needs a server, and how to avoid one

Emailing someone their results does need to leave the browser. You do **not** need a backend for it. A **Cloudflare Worker** — about thirty lines, free tier, deployed in minutes — accepts the results and sends the mail. That's the entire server-side footprint of eight tools.

Build the capture step at the same time as the tool. I didn't. For months, every one of my tools asked for an email *optionally*. Someone could complete a quiz that told them they needed exactly what I sell, and leave without me ever knowing they existed. The hard part — the tool — was done. The easy part was missing.

Prompt to start from

"Build a single self-contained HTML page: a 25-question operational health assessment across five categories. Each question rates 0–2. Show a live overall score and a per-category breakdown as the user answers. Persist answers in localStorage. Add a print stylesheet that produces a clean one-page report. No external dependencies, no server calls."

PHASE 13 · NEW IN THIS EDITION

Add a blog without adding a platform

I spent a while assuming a blog meant another subscription, or a second site, or moving everything to something heavier. It doesn't. **GitHub Pages runs Jekyll natively**, at no cost, on the repo you already have.

What you add

- `_config.yml` — your site title, and a permalink pattern. Mine is `/blog/:year/:title/`, which produces clean, durable URLs.
- `_posts/` — one Markdown file per post, named `YYYY-MM-DD-title.md`. Write in Markdown, not HTML.
- `_layouts/post.html` — one template every post is poured into. Build it once, with your nav, footer, analytics and security headers, and every future post inherits all of it.

Push a Markdown file, and a styled page appears at a clean URL. That's the whole workflow.

Then add the feed

One plugin line gives you a real RSS feed. It costs nothing, it makes your writing syndicable, and — increasingly — it is how systems other than browsers find your work.

The trap I fell into: I built the blog, wrote seven posts, and never added any of it to my sitemap. Seven pieces of writing, invisible to search, for two months. Jekyll builds the pages; it does not tell Google they exist. That is still your job.

Why this beats a hosted platform for most people: the posts are files in your repo. No export, no lock-in, no migration the day the platform changes its pricing. And every post automatically carries the same security headers and analytics as the rest of your site, because it inherits the same layout.

CLOSING · NEW IN THIS EDITION

What drifts, and how to catch it

The first edition ended at launch. Everything I have learned since is about what happens after.

Owning your site means nobody else is maintaining it. Wix at least nagged you. A static site fails silently — it will serve a mistake perfectly, at speed, with a valid certificate, forever. Here is exactly what I found on my own site three months in.

WHAT DRIFTED	HOW BAD
An expired banner — a "vote by June 19" call to action	Still on sixteen pages in September. The replacement was on one.
A duplicate page — two versions of my tools page	Both in the sitemap, competing with each other for the same ranking.
A finished tool nothing linked to	Built, working, invisible. Zero visitors, ever.
A live page of placeholders — <code>[Scope summary]</code> , <code>\$(XX,XXX)</code>	Public, on a page titled "Transparency & Accountability."
Two analytics vendors doing one job	Found only by reading my own security policy.

Not one of those was a technical failure. Every page loaded correctly. They were *editorial* failures, and no tool alerts you to them.

The quarterly audit, in five questions

- **What has a date on it?** Banners, "current" lists, years in footers. Anything with a date is a future mistake.
- **What links to nothing, and what does nothing link to?** Orphans in both directions.
- **Does the sitemap match the site?** Compare it to your actual file list, not your memory.
- **What third-party scripts are loading, and do I still use all of them?** Your CSP is the honest inventory.
- **Is there a placeholder anywhere?** Search your whole site for `[` and `TODO` and `Lorem`.

Claude can run all five against a copy of your repo in a few minutes and hand you a list. That is a far better use of a co-pilot than writing another page.

What I'd do differently: build the email capture at the same time as the thing it captures for. Add every new page to the sitemap the day it ships. And put the audit in the calendar, because a site nobody audits is a site slowly telling people something that stopped being true.

Let ToyBox build it with you.

You can do every phase in this playbook yourself — that's the point of writing it down. If you'd rather not, that's what I do for a living.

KATOYA RAQUELL PALMER

Founder & Principal Consultant, ToyBox Consulting & Management, LLC

katoya@toyboxconsulting.net

toyboxconsulting.net · katoyapalmer.com · katoyapalmer.substack.com